

ITEC610 – INTRODUCTION TO DATA SCIENCE WITH PYTHON

Assessment 3: Data processing and exploration project

Semester 2, 2024

Submitted to:

Vu Nguyen

Submitted by:

Amir Tamang

Student ID: S00393664

Email: S00393664@myacu.edu.au

Video Presentation link: [S00303664_Amir_Tamang ITEC610_AT3 - YouTube](#)

Date of Submission: 28/10/2024

Exploring BRFSS data

Introduction

ITEC610_assessment3_data.csv is a project in the class of ITEC610 in the methods of data analysis and visualization. This dataset most probably originated from some survey or study that measured participants on their physical character, height, and weight to analyze health or fitness. Such information might be used in studying the pattern through body measurements over time, assessing weight change, or looking for patterns that relate height and weight among a set of individuals in a community. (Stack Exchange, 2022).

This dataset contains:

Row = 414508

Columns = 7

Each row typically represents an individual participant, and the columns represent different variables, such as:

1. The first column variable is empty, it is the column for serial number.
2. weight2: Current weight of the participant.
3. wtyr ago: The participant's weight was a year ago.
4. Wtkg2: Shows the roundup value of current weight.
5. htm3: Height of the participant.
6. Sex: 1 and 2 determine the value of gender identity.

These variables allow for the analysis of changes in weight over time and how these changes relate to height, showing details of individuals and populations.

Section 1: Summary Statistics Analysis

Task 1

```
import pandas as pd # Importing the pandas library for data manipulation
import matplotlib.pyplot as plt # Importing matplotlib for data visualization

# Load the dataset from the specified CSV file
data = pd.read_csv('/content/ITEC610_assessment3_data.csv')

# Calculate summary statistics for the specified columns: weight2, wtyrigo (weight a year ago), and htm3 (height)
summary = data[['weight2', 'wtyrigo', 'htm3']].describe()
print("Summary Statistics:\n", summary) # Print the summary statistics for verification

# Define category labels for the plot
categories = ['Current_weight', 'Weight_a_year_ago', 'Height']

# Extract statistical values for each category
min_vals = summary.loc['min'].values # Minimum values
max_vals = summary.loc['max'].values # Maximum values
mean_vals = summary.loc['mean'].values # Mean values
median_vals = summary.loc['50%'].values # Median values (50th percentile)
std_vals = summary.loc['std'].values # Standard deviation values
percentile_25 = summary.loc['25%'].values # 25th percentile values
percentile_75 = summary.loc['75%'].values # 75th percentile values

# Plotting summary statistics
plt.figure(figsize=(8, 6)) # Set the figure size
plt.scatter(categories, max_vals, markers='v', color='black', label='max', s=100) # Maximum values
```

Fig.1. Task 1 code.

1. Importing Libraries and loading data:

- We bring in two tools: `pandas` (to handle data) and `matplotlib.pyplot` (to make graphs).
- We read a file named `ITEC610\_assessment3\_data.csv` and store its content in a variable called `data`.

2. Calculating Statistics:

- We look at three columns in our data: `weight2` (current weight), `wtyrigo` (weight a year ago), and `htm3` (height).
- use `describe()` to get important numbers like the average (mean), smallest (min), largest (max), and other statistics for these columns.
- print these numbers to check them.

3. Setting Up Categories:

- We create a list of names for our categories: ``['Current_weight', 'Weight_a_year_ago', 'Height']``.

4. Getting Values for Plotting:

- We pull out specific numbers from our statistics:
 - Minimum values
 - Maximum values
 - Mean (average) values
 - Median (middle value)
 - Standard deviation (how spread out the numbers are)
 - 25th percentile (value below which 25% of the data falls)
 - 75th percentile (value below which 75% of the data falls)

5. Making the Plot:

- set the size of our graph.
- plot different points for each category using different shapes and colors:
 - Triangles for min and max values
 - Crosses for mean values
 - Stars for median values
 - Green triangles for mean plus/minus standard deviation
 - Orange arrows for 25th and 75th percentiles

S00393664

- add a legend to explain what each shape and color means.
- give our graph a title and label the y-axis.
- adjust the layout to make everything fit nicely.

Output:

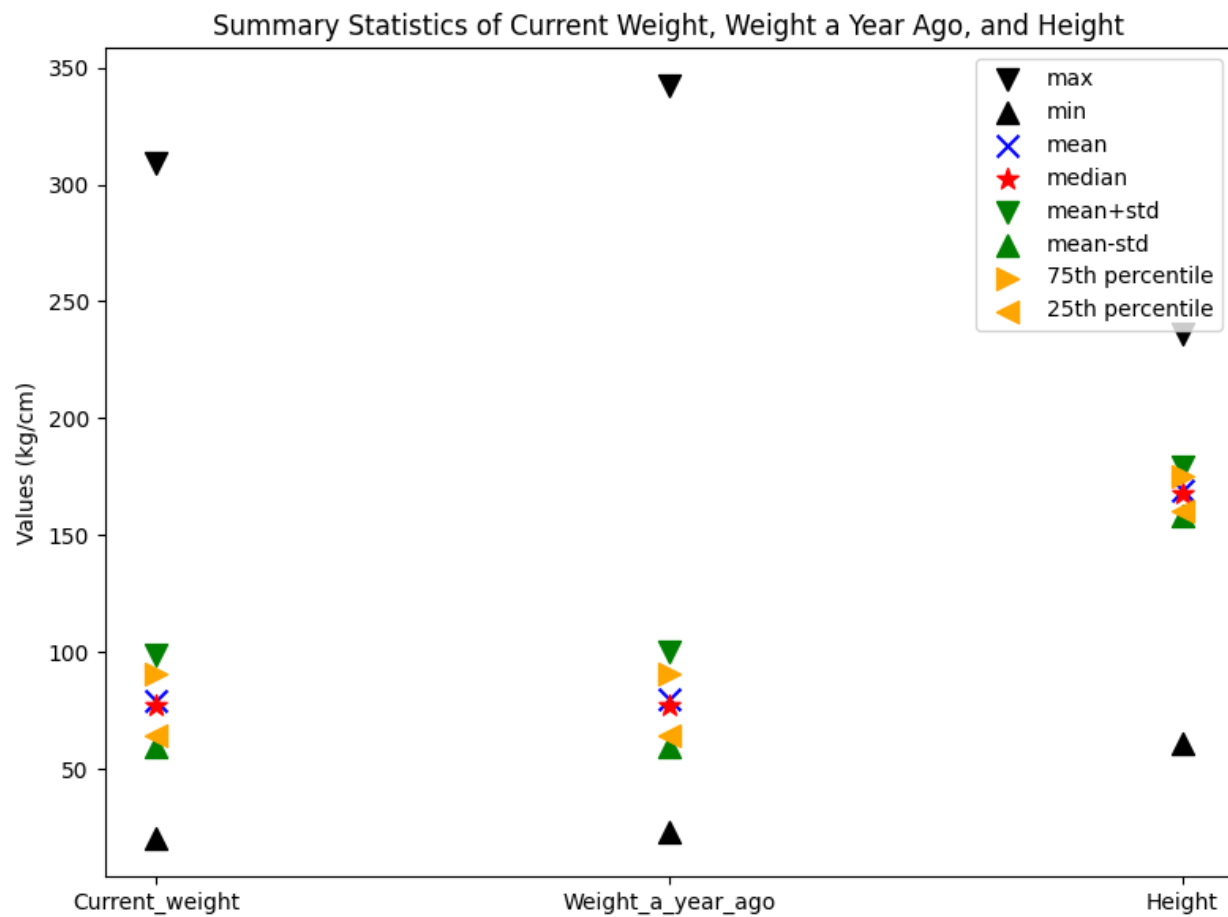


Fig.2. Task 1 Output.

The chart displays the mean and standard deviation for weight2, wtyr ago, and htm3, showing each variable's average values.

Section 2: Correlations Analysis

Task 2

```
# Calculate the change in weight by subtracting the weight from a year ago from the current weight
data['weight_change'] = data['weight2'] - data['wtyrago']

# Create a dictionary to store the correlations between weight change and other variables
correlations = {
    'weight2': data['weight_change'].corr(data['weight2']), # Correlation between weight change and current weight
    'wtyrago': data['weight_change'].corr(data['wtyrago']), # Correlation between weight change and weight a year ago
    'age': data['weight_change'].corr(data['age'])           # Correlation between weight change and age
}

# Create a scatter plot to visualize the relationship between weight change and current weight
plt.scatter(data['weight_change'], data['weight2']) # X-axis: Weight change, Y-axis: Current weight
plt.title('Weight Change vs Current Weight') # Add a title to the plot
plt.xlabel('Weight Change') # Label the X-axis
plt.ylabel('Current Weight') # Label the Y-axis
plt.show() # Display the plot

# Create another scatter plot to visualize weight change against weight from a year ago
plt.scatter(data['weight_change'], data['wtyrago']) # X-axis: Weight change, Y-axis: Weight a year ago
plt.title('Weight Change vs Weight a Year Ago') # Add a title to the second plot
plt.xlabel('Weight Change') # Label the X-axis
plt.ylabel('Weight a Year Ago') # Label the Y-axis
plt.show() # Display the second plot

# Create a third scatter plot to visualize weight change against age
```

Fig.3. Task 2 code.

- To calculate weight change, the code computes a new column called `weight_change` by subtracting (`weight2`-the current weight-from `wtyrago`-weight from a year ago). Then, the correlations between the variable `weight_change` and three other variables, namely `weight2`, `wtyrago`, and `age` are computed and placed in a dictionary identified as `correlations`.
- Relationships are observed by creating scatter plots.
 - **Current weight (`weight2`)**

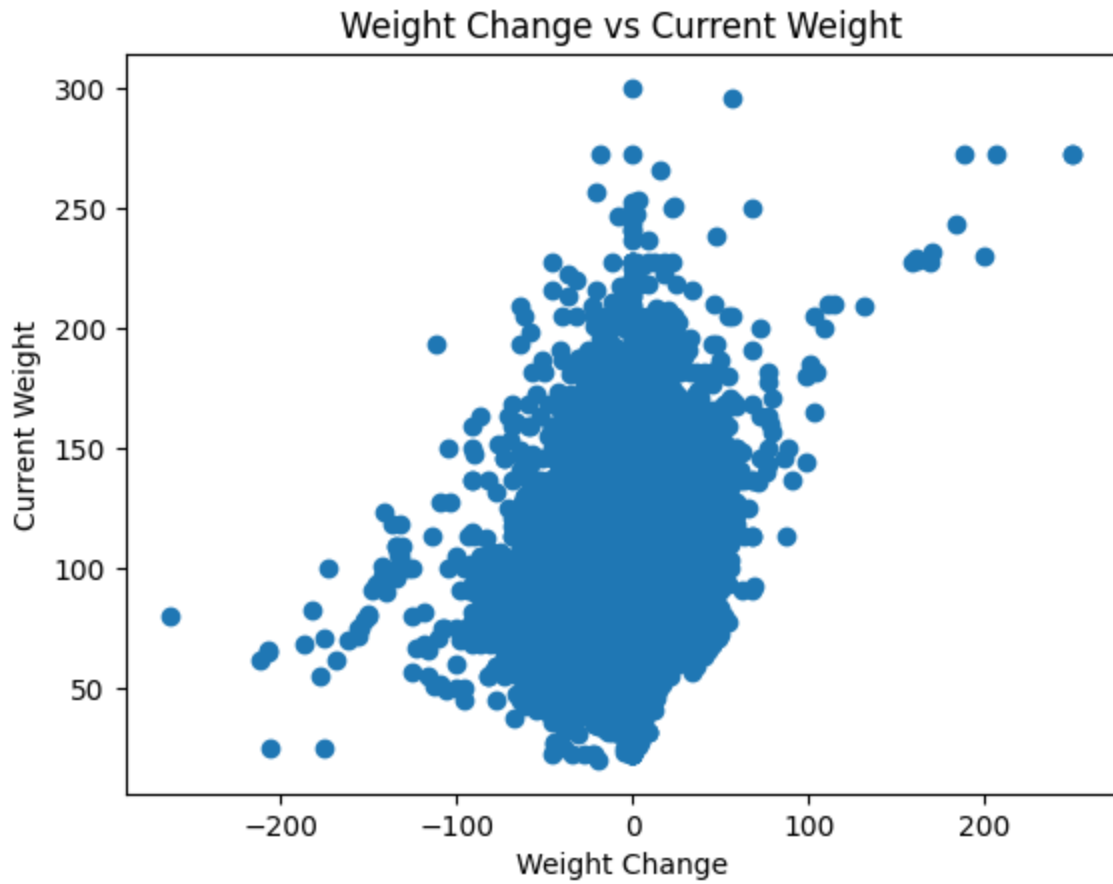


Fig.4. Weight change VS Current weight

- **Weight a year ago (wtyr ago)**

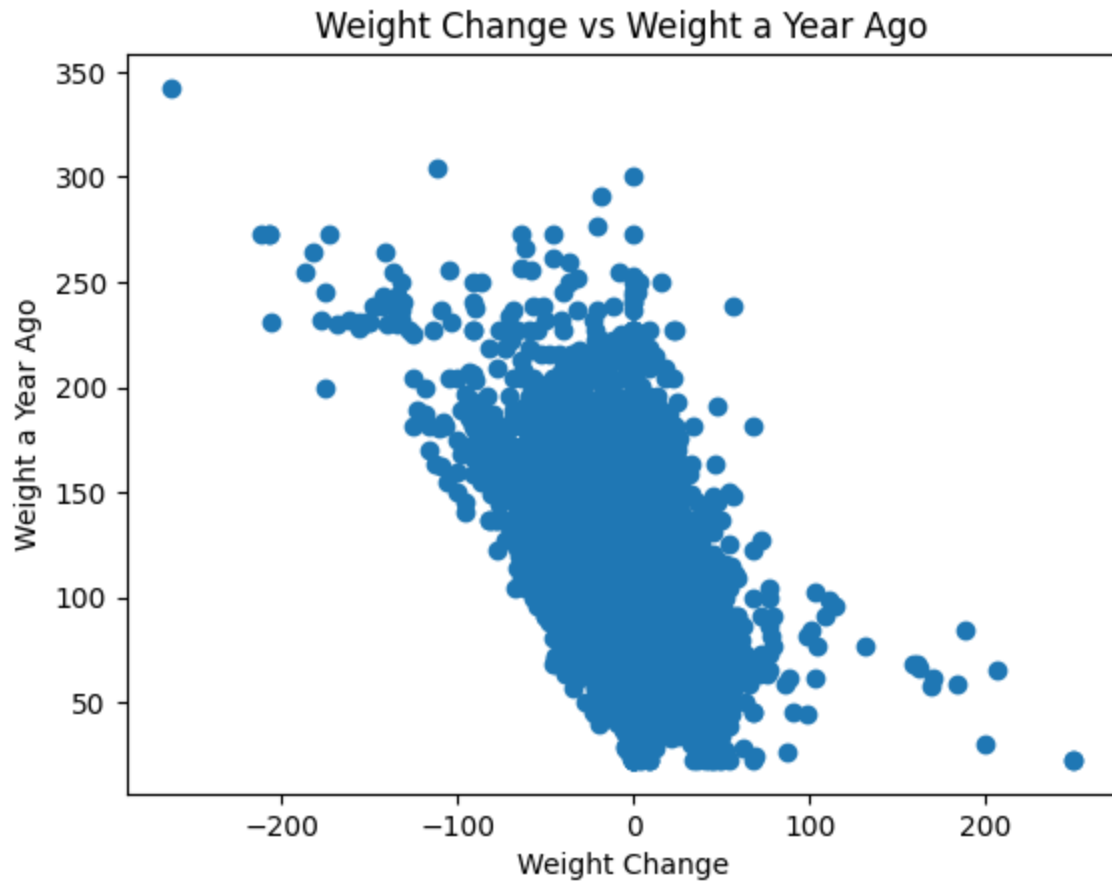


Fig.5. Weight change vs. weight a year ago.

○ Age

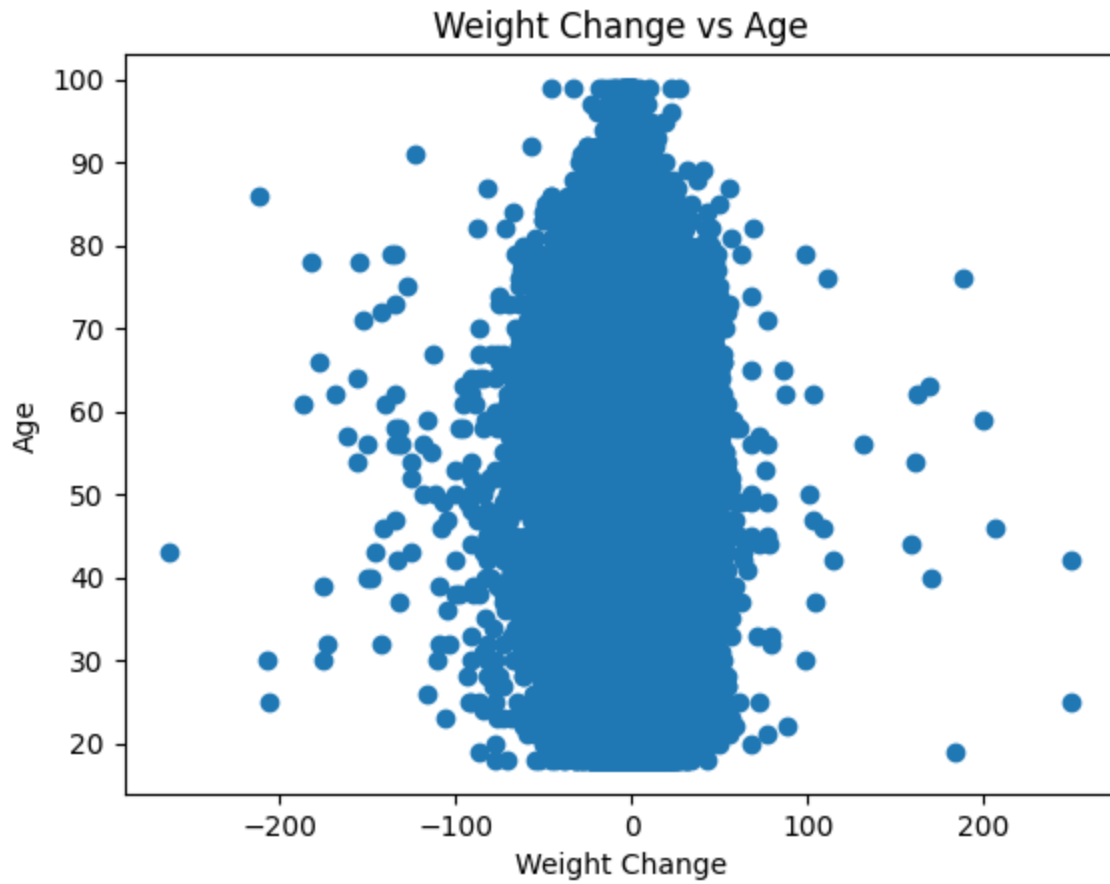


Fig.6. Weight Change VS Age.

Section 3: Significant analysis

Task 3

```
from sklearn.linear_model import LinearRegression # Import the LinearRegression class from sklearn

# Data: Defining house sizes (in square feet) and their corresponding prices (in dollars)
house_sizes = [[1500], [2000], [2500], [3000], [3500]] # House sizes as a 2D list (required for sklearn)
house_prices = [250000, 300000, 330000, 360000, 390000] # List of house prices

# Model: Creating an instance of the LinearRegression model
model = LinearRegression()

# Fitting the model with the provided data (training the model)
model.fit(house_sizes, house_prices)

# Prediction: Using the trained model to predict the price of a house with 2500 sqft
predicted_price = model.predict([[2500]]) # Make a prediction for a 2500 sqft house

# Output: Displaying the predicted price in a formatted string
print(f'Predicted price for 2500 sqft house: ${predicted_price[0]:.2f}') # Print the predicted price with two decimal places
```

Fig.7. Task 3 code.

- The LinearRegression class from Sklearn is loaded to perform predictive modeling.
- The dataset contains house sizes in square feet and prices. These are pairs of data points, one each for prices and house sizes.
- A linear regression is instantiated with house_sizes as the input feature and house_prices as the target variable, and this regressor is then trained using the fit() function.
- The predict() method is then used to estimate the price for a property of 2,500 square feet in area after the training of the model.

Output:

```
Predicted price for 2500 sqft house: $326000.00
```

Fig.8. Task 3 output.

Section 4: Conclusion

The dataset indeed illustrates that persons with higher historical weights are likely to have higher changes over time, which is evoked by the strong correlation between weight change and weight from a year ago. Current weight and weight change do present some positive correlation; in other words, those individuals who have higher weights now also gained more weight recently.

Whereas the variation in weight does not change drastically with age, this would imply that lifestyle or health behavior matters more. Besides giving an example of how statistical techniques and predictive modeling can be used to analyze patterns and make forecasts, the research also brings into focus the importance of historical weight data in understanding weight swings.

S00393664

Reference

Stack Exchange, 2022. Age, Weight, and Height dataset [WWW Document]. URL <https://opendata.stackexchange.com/questions/7793/age-weight-and-height-dataset> (accessed 10.28.24).